

Guide Utilisateur - Thumalien

Systeme de Detection de Fake News Bilingue FR/EN

1. Presentation

Thumalien est un systeme d'analyse automatisee de contenus textuels pour la detection de fake news. Il traite les posts publies sur le reseau social Bluesky en francais et en anglais, et fournit pour chaque texte un score de credibilite, une emotion dominante et une classification fiable/suspect.

Le systeme repose sur un pipeline NLP bilingue (V1.5) combinant des features TF-IDF, linguistiques et emotionnelles pour atteindre un F1-score superieur a 0.98 dans les deux langues.

2. Prerequis

- Python 3.13+
 - pip (gestionnaire de paquets)
 - MongoDB (optionnel - le dashboard fonctionne en mode demo sans base de donnees)
 - Environ 2 Go d'espace disque (modeles + donnees)
-

3. Installation

```
# Cloner le depot
git clone https://github.com/azelbanks/projet_etude.git
cd projet_etude

# Installer les dependances
pip install -r requirements.txt

# Verifier l'installation
python -c "from src.pipeline.expert_detector import ExpertFakeNewsDetector; print('OK')"
```

4. Structure du projet

```
projet_etude/
|-- dashboard/
|   |-- app.py           # Dashboard Streamlit (interface principale)
|-- data/
|   |-- training/        # Datasets d'entrainement (non versiones)
|   |   |-- Fake.csv     # ISOT Fake News (anglais)
|   |   |-- True.csv     # ISOT True News (anglais)
|   |   |-- kaggle_fr/   # Kaggle FrenchFakeNewsDetector
|   |   |-- train.csv    # Emotions EN (entrainement)
```

```
| | |-- training.csv          # Emotions FR (entrainement)
|-- docs/
| |-- guide_utilisateur.md   # Ce fichier
| |-- architecture.png       # Schema d'architecture du pipeline
| |-- gantt_planning.png     # Diagramme de Gantt retrospectif
|-- models/
| |-- model_expert.pkl        # Modele LogReg bilingue
| |-- tfidf_expert.pkl        # Vectoriseur TF-IDF (30K features)
| |-- metrics_expert.pkl      # Metriques d'entrainement
| |-- emotion_bilingual.pt    # MLP PyTorch (7 emotions)
| |-- emotion_vocab_bilingual.pickle
| |-- emotion_label_encoder_bilingual.pickle
|-- notebooks/
| |-- 00_Audit_Qualite_Donnees.ipynb
| |-- 01_Exploration_Bluesky.ipynb
| |-- 02_Analyse_Emotions_MLP.ipynb
| |-- 03_Mise_a_jour_Quotidienne.ipynb
| |-- 04_Modele_Avance_RoBERTa.ipynb
| |-- 05_Detection_Expert_Bilingue.ipynb
| |-- 06_Documentation_Technique.ipynb
|-- src/
| |-- pipeline/
| | |-- expert_detector.py    # Pipeline complet (classes principales)
| |-- collection/
| | |-- collect_bluesky.py    # Collecte AT Protocol
| |-- app/
| | |-- main.py               # Point d'entree applicatif
|-- .streamlit/
| |-- config.toml             # Theme dark + config serveurur
|-- docker-compose.yml
|-- dockerfile
|-- requirements.txt
|-- emissions.csv             # Bilan carbone CodeCarbon
```

5. Lancer le dashboard

```
# Depuis la racine du projet
streamlit run dashboard/app.py
```

Le dashboard s'ouvre automatiquement dans le navigateur a l'adresse <http://localhost:8501>.

Mode demo : si MongoDB n'est pas connecte, le dashboard affiche des donnees d'exemple (15 posts FR/EN) avec un bandeau informatif.

6. Pages du dashboard

6.1. Vue Globale

La page d'accueil presente une vision synthetique de l'ensemble des posts analyses :

- Metriques cles : nombre total de posts, pourcentage de posts fiables, score de credibilite moyen, repartition FR/EN.

- Profil émotionnel : radar chart des 7 émotions moyennes sur l'ensemble du corpus (colère, dégoût, joie, neutre, peur, surprise, tristesse).
- Fiabilité par langue : diagramme en barres horizontales comparant les posts fiables et suspects pour le français et l'anglais.
- Tableau des posts : liste des derniers posts analysés avec leur texte (tronqué), langue, label et score de crédibilité.

6.2. Analyse en temps réel

Cette page permet d'analyser un texte en temps réel :

- Collez un texte (article, post Bluesky, tweet, ou tout texte FR/EN) dans la zone de saisie.
- Cliquez sur le bouton Analyser.
- Le système retourne :
 - Un score de crédibilité (jauge 0 à 1) : 0 = probablement faux, 1 = probablement fiable.
 - Un verdict : FIABLE (vert) ou SUSPECT (rouge).
 - L'émotion dominante détectée avec sa probabilité.
 - Un radar chart détaillé des 7 probabilités émotionnelles.
 - La langue détectée automatiquement (FR ou EN).

Interprétation du score :

- Score > 0.7 : le texte présente des caractéristiques de contenu fiable.
- Score entre 0.4 et 0.7 : zone d'incertitude, vérification manuelle recommandée.
- Score < 0.4 : le texte présente des marqueurs de désinformation.

> Avertissement : le score est un indicateur probabiliste basé sur des patterns statistiques. Il ne constitue pas une vérification factuelle et ne doit pas être utilisé comme seul critère de décision.

6.3. Métriques & Transparence

Cette page fournit les indicateurs de performance et de conformité :

- Ablation study : tableau et graphique des F1-scores pour les 7 conditions expérimentales testées (EN seul, FR seul, bilingue, bilingue + émotions, etc.).
- Bilan carbone : émissions CO2 totales du projet mesurées par CodeCarbon.
- Roadmap : les 4 versions planifiées du pipeline (V1 à V3) avec leurs objectifs.
- Conformité : fiches RGPD et IA Act résumant les mesures de conformité.

7. Utiliser le pipeline en Python

Le pipeline peut être utilisé directement en Python sans le dashboard :

```
import sys
sys.path.insert(0, 'src')

from pipeline.expert_detector import ExpertFakeNewsDetector
import pandas as pd

# Charger le modèle
detector = ExpertFakeNewsDetector(model_dir='models', use_emotions=True)
```

```

detector.load(suffix='expert')

# Analyser des textes
textes = pd.Series([
    "Le CNRS publie une etude sur le climat.",
    "BREAKING: Secret labs control your mind with 5G!!!",
    "La BCE maintient ses taux directeurs.",
])

resultats = detector.predict(textes)
print(resultats[['text', 'language', 'prediction_label', 'ai_score_credibility']])

```

Colonnes retournees par predict() :

Colonne	Description
text	Texte original
language	Langue detectee (fr, en, other)
prediction_label	0 = Fiable, 1 = Suspect
ai_score_credibility	Probabilite de fiabilite (0 a 1)
ai_analysis_log	Log d'analyse lisible (ex: "[FR] Fiable (credibilite: 89%)")

8. Utiliser l'analyse d'émotions

```

from pipeline.expert_detector import EmotionFeatureExtractor

emo = EmotionFeatureExtractor(model_dir='models')
emo.load()

probas = emo.get_emotion_features([
    "Je suis tellement heureux de cette nouvelle !",
    "This is absolutely terrifying news.",
])

# probas.shape = (2, 7) - 7 probabilites par texte
# Classes : colere, degout, joie, neutre, peur, surprise, tristesse
print(probas)

```

9. Notebooks

Les notebooks documentent chaque etape du projet et sont executes sequentiellement :

Notebook	Description	Sortie
00	Audit qualite des donnees d'entrainement	Statistiques, doublons, distributions
01	Exploration des posts Bluesky collectes	Visualisations, wordclouds, distributions temporelles
02	Entrainement du MLP emotions bilingue (PyTorch)	emotion_bilingual.pt + vocabulaire + label encoder

03	Pipeline de mise a jour quotidienne	Collecte + inference sur posts recents
04	Prototype RoBERTa avance	Exploration fine-tuning (non deploye)
05	Pipeline expert bilingue + ablation study (7 conditions)	model_expert.pkl + tfidf_expert.pkl + metriques
06	Documentation technique complete	Limites, roadmap, PRA/PCA, Green IT, conformite

Pour re-executer un notebook :

```
jupyter nbconvert --to notebook --execute notebooks/05_Detection_Expert_Bilingue.ipynb \
--output 05_Detection_Expert_Bilingue.ipynb --ExecutePreprocessor.timeout=600
```

10. Deploiement Docker

Le projet inclut un docker-compose.yml pour le deploiement complet :

```
# Lancer l'ensemble de la stack
docker-compose up -d

# Verifier les services
docker-compose ps

# Consulter les logs
docker-compose logs -f
```

Services :

- MongoDB : stockage des posts collectes et des resultats d'analyse.
- Collecteur Bluesky : script de collecte automatisee via le protocole AT.
- Dashboard Streamlit : interface web de visualisation (port 8501).

11. Configuration

Variables d'environnement (fichier .env)

```
BLUESKY_HANDLE=votre_handle.bsky.social
BLUESKY_APP_PASSWORD=votre_app_password
MONGODB_URI=mongodb://mongodb:27017/
```

Theme du dashboard (.streamlit/config.toml)

Le theme dark est configure par default. Pour modifier les couleurs, editez .streamlit/config.toml :

```
[theme]
primaryColor = "#00D4FF"
backgroundColor = "#0E1117"
secondaryBackgroundColor = "#1A1F2E"
textColor = "#E0E0E0"
```

12. FAQ

Q : Le dashboard affiche "Mode demo". Comment connecter MongoDB ?

R : Lancez MongoDB (via Docker ou en local sur le port 27017) et assurez-vous que la base thumalien_db contient une collection raw_posts. Le dashboard se connecte automatiquement au démarrage.

Q : Comment re-entraîner le modèle avec de nouvelles données ?

R : Exécutez le notebook 05 (05_Detection_Expert_Bilingue.ipynb). Il recharge les CSV depuis data/training/, re-entraîne le modèle et sauvegarde les fichiers .pkl dans models/.

Q : Le modèle classe mal un texte. Que faire ?

R : Le modèle est entraîné sur des articles de presse et peut mal généraliser sur des textes courts ou atypiques (posts de 10 mots, memes, satire). Le score doit être interprété comme un indicateur, pas comme un verdict. Consultez la section "Limites" du notebook 06.

Q : Puis-je ajouter d'autres langues ?

R : La V1.5 supporte uniquement le français et l'anglais. Les textes dans d'autres langues sont routés vers le pipeline anglais par défaut. La V2 (sentence-transformers multilingue) étendra le support à 50+ langues.

Q : Quel est le coût carbone d'une prédiction ?

R : Une prédiction sur un batch de 1 000 textes émet moins de 0.001 g de CO2. L'entraînement complet émet environ 0.01 g de CO2 (moins qu'un email).

13. Support

- Code source : github.com/azelbanks/projet_etude
- Documentation technique : notebook 06
- Bugs et suggestions : ouvrir une issue sur le dépôt GitHub

Thumalien v1.5 - Pipeline bilingue FR/EN - Mastere Big Data, Sup de Vinci